

CYBERSECURITY FINAL PROJECT

Title: Secure Registration and Login System

IT322 – Information Assurance and Security 1

Members:

Acido, Elishama

Batawang, Homer

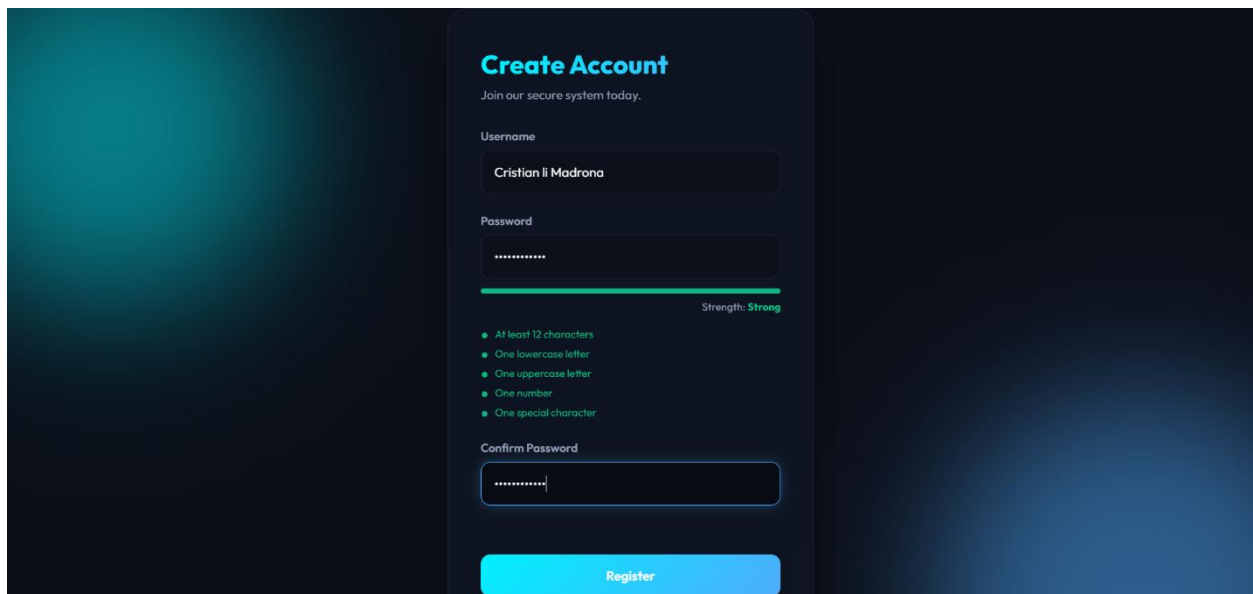
Labad, RodSteven

Madrona, Cristian li

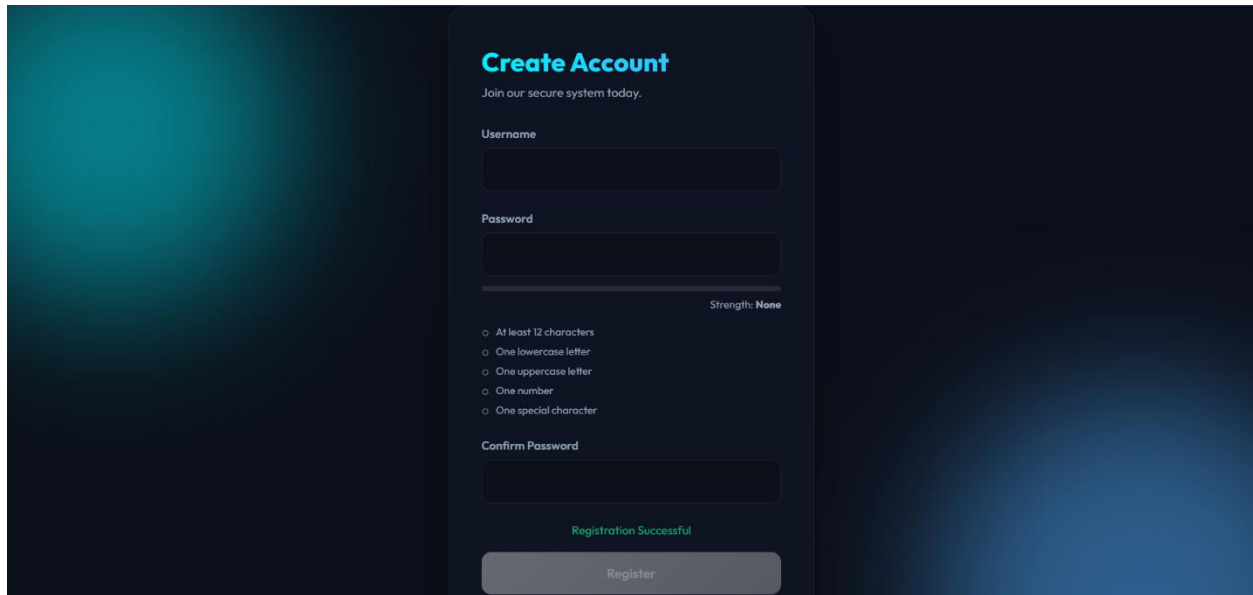
A. Screenshots

Registration Module

Password meter

A screenshot of a web application's registration interface. The background is dark with a teal-to-blue gradient on the left. The registration form is centered and has a dark blue background. It features the title 'Create Account' in teal, followed by the subtitle 'Join our secure system today.' in white. The form includes three input fields: 'Username' with the text 'Cristian li Madrona', 'Password' with masked characters '*****', and 'Confirm Password' with masked characters '*****'. Below the password field is a green progress bar and the text 'Strength: Strong'. A list of password requirements is shown with green bullet points: 'At least 12 characters', 'One lowercase letter', 'One uppercase letter', 'One number', and 'One special character'. At the bottom of the form is a bright blue 'Register' button.

Successful registration

A dark-themed 'Create Account' form. The title 'Create Account' is in teal. Below it is the subtitle 'Join our secure system today.' The form contains three input fields: 'Username', 'Password', and 'Confirm Password'. The 'Password' field has a strength indicator below it showing 'Strength: None' and a list of requirements: 'At least 12 characters', 'One lowercase letter', 'One uppercase letter', 'One number', and 'One special character'. A green message 'Registration Successful' is displayed above a grey 'Register' button.

Create Account

Join our secure system today.

Username

Password

Strength: None

- At least 12 characters
- One lowercase letter
- One uppercase letter
- One number
- One special character

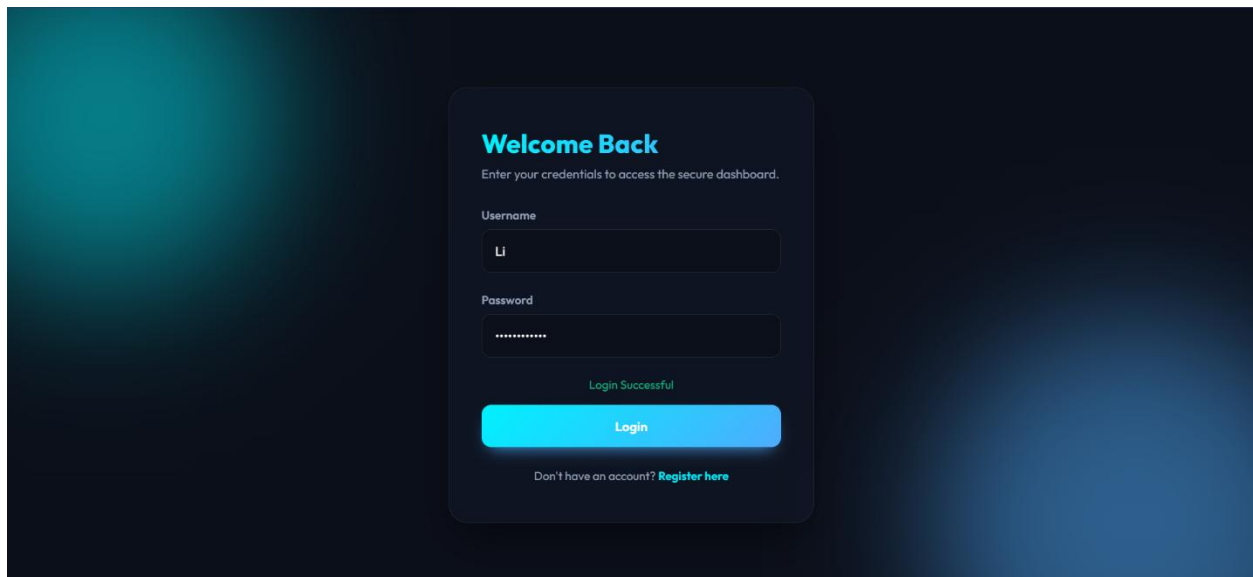
Confirm Password

Registration Successful

Register

Login Module

Successful login

A dark-themed 'Welcome Back' login form. The title 'Welcome Back' is in teal. Below it is the subtitle 'Enter your credentials to access the secure dashboard.' The form contains two input fields: 'Username' (with the text 'Li') and 'Password' (with masked characters '*****'). A green message 'Login Successful' is displayed above a bright blue 'Login' button. At the bottom, there is a link: 'Don't have an account? [Register here](#)'.

Welcome Back

Enter your credentials to access the secure dashboard.

Username

Li

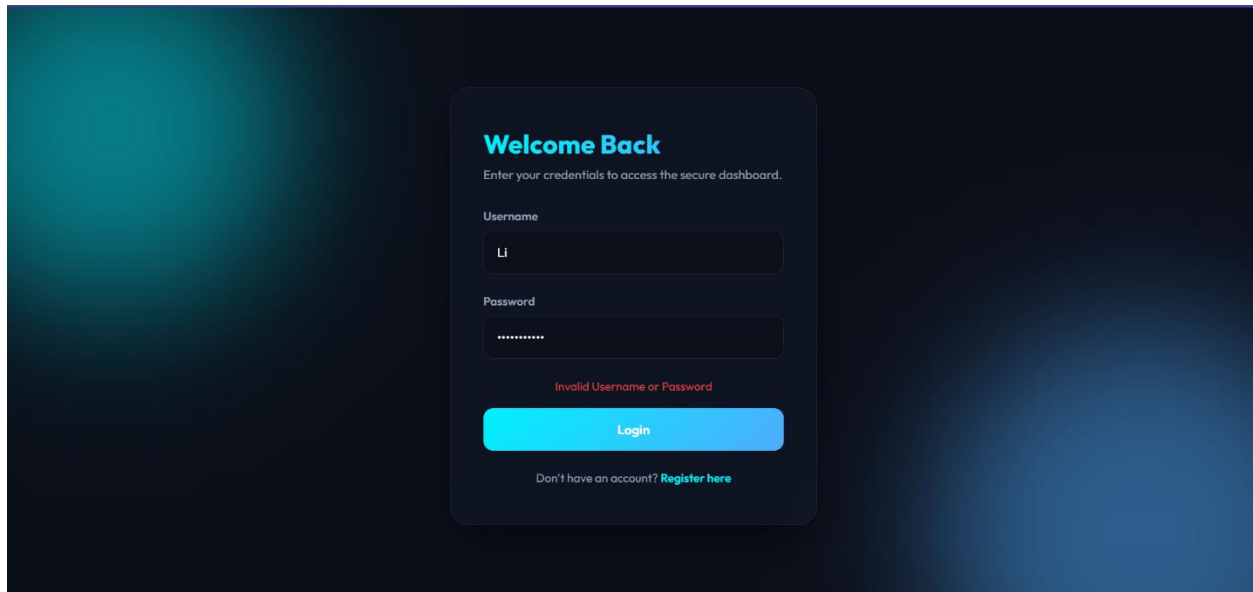
Password

Login Successful

Login

Don't have an account? [Register here](#)

Failed login attempt



A login form titled "Welcome Back" with a subtitle "Enter your credentials to access the secure dashboard." It features two input fields: "Username" containing "Li" and "Password" containing "*****". Below the password field is a red error message: "Invalid Username or Password". At the bottom is a blue "Login" button and a link "Don't have an account? Register here".

Welcome Back

Enter your credentials to access the secure dashboard.

Username

Li

Password

Invalid Username or Password

Login

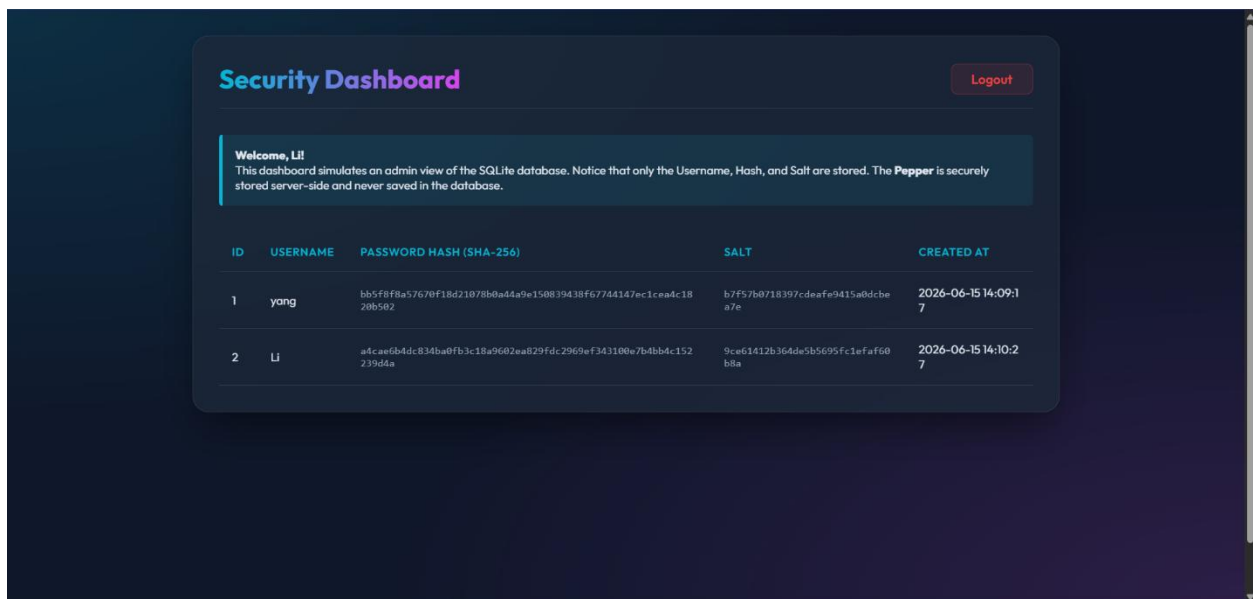
Don't have an account? [Register here](#)

Database/Table

Username

Password Hash

Salt



A screenshot of a "Security Dashboard" with a "Logout" button in the top right. A message box says "Welcome, Li! This dashboard simulates an admin view of the SQLite database. Notice that only the Username, Hash, and Salt are stored. The Pepper is securely stored server-side and never saved in the database." Below is a table with columns: ID, USERNAME, PASSWORD HASH (SHA-256), SALT, and CREATED AT. It contains two rows of user data.

ID	USERNAME	PASSWORD HASH (SHA-256)	SALT	CREATED AT
1	yang	bb5f8f8a57670f18d21078b0a44a9e150839438f67744147ec1ced4c1820b502	b7f57b0718397cdea9415a0dchea7e	2026-06-15 14:09:17
2	Li	a4caeb6b4dc834ba0fb3c18a9602ea829f4dc2969ef343100e7b4bb4c1522330daa	9ce61412b364de5b5695fc1efaf60b8a	2026-06-15 14:10:27

B. Source Code

1. source_code/index.html

(The main entry point with the Registration and Login forms)

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Secure Authentication</title>

  <link
href="https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;600;800&display=swap"
rel="stylesheet">

  <link rel="stylesheet" href="style.css">

</head>

<body>

  <div class="background-elements">

    <div class="blob blob-1"></div>

    <div class="blob blob-2"></div>

  </div>

  <div class="auth-container">

    <!-- Login Form -->

    <div class="form-box" id="login-box">

      <h2>Welcome Back</h2>

      <p class="subtitle">Enter your credentials to access the secure dashboard.</p>

      <form id="login-form">

        <div class="input-group">
```

```
<label for="login-username">Username</label>

<input type="text" id="login-username" name="username" required autocomplete="off">
</div>

<div class="input-group">

  <label for="login-password">Password</label>

  <input type="password" id="login-password" name="password" required>
</div>

<div class="message" id="login-message"></div>

<button type="submit" class="btn-primary">Login</button>
</form>

<div class="toggle-link">

  Don't have an account? <span onclick="toggleForms()">Register here</span>
</div>
</div>
```

<!-- Registration Form -->

```
<div class="form-box hidden" id="register-box">

  <h2>Create Account</h2>

  <p class="subtitle">Join our secure system today.</p>

  <form id="register-form">

    <div class="input-group">

      <label for="reg-username">Username</label>

      <input type="text" id="reg-username" name="username" required autocomplete="off">
    </div>

    <div class="input-group">

      <label for="reg-password">Password</label>

      <input type="password" id="reg-password" name="password" required>

      <div class="password-meter-container">

        <div class="meter-bar" id="meter-bar"></div>
      </div>
    </div>
  </form>
</div>
```

</div>

<div class="strength-text" id="strength-text">Strength: Weak</div>

<ul class="requirements-list">

<li id="req-length">At least 12 characters

<li id="req-lower">One lowercase letter

<li id="req-upper">One uppercase letter

<li id="req-digit">One number

<li id="req-symbol">One special character

</div>

<div class="input-group">

<label for="reg-confirm-password">Confirm Password</label>

<input type="password" id="reg-confirm-password" name="confirm_password" required>

</div>

<div class="message" id="reg-message"></div>

<button type="submit" class="btn-primary" id="btn-register" disabled>Register</button>

</form>

<div class="toggle-link">

Already have an account? Login here

</div>

</div>

</div>

<script src="script.js"></script>

</body>

</html>

2. source_code/style.css

(The glassmorphism, vibrant colors, and responsive premium design)

```
:root {  
  --bg-dark: #0b0f19;  
  --glass-bg: rgba(17, 24, 39, 0.6);  
  --glass-border: rgba(255, 255, 255, 0.08);  
  --accent-cyan: #00f2fe;  
  --accent-magenta: #4facfe;  
  --accent-success: #10b981;  
  --accent-warning: #f59e0b;  
  --accent-error: #ef4444;  
  --text-primary: #f8fafc;  
  --text-secondary: #94a3b8;  
}
```

```
* {  
  box-sizing: border-box;  
  margin: 0;  
  padding: 0;  
}
```

```
body {  
  font-family: 'Outfit', sans-serif;  
  background-color: var(--bg-dark);  
  color: var(--text-primary);  
  min-height: 100vh;  
  display: flex;  
  justify-content: center;  
  align-items: center;
```

```
position: relative;
overflow: hidden;
}
```

```
/* Dynamic Background Orbs */
```

```
.background-elements {
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  z-index: 0;
  overflow: hidden;
  pointer-events: none;
}
```

```
.blob {
  position: absolute;
  border-radius: 50%;
  filter: blur(80px);
  opacity: 0.5;
  animation: float 10s infinite ease-in-out alternate;
}
```

```
.blob-1 {
  width: 400px;
  height: 400px;
  background: var(--accent-cyan);
  top: -100px;
```



```
    left: -100px;
}
```

```
.blob-2 {
    width: 500px;
    height: 500px;
    background: var(--accent-magenta);
    bottom: -150px;
    right: -100px;
    animation-delay: -5s;
}
```

```
@keyframes float {
    0% { transform: translate(0, 0) scale(1); }
    100% { transform: translate(30px, 50px) scale(1.1); }
}
```

/* Container & Forms */

```
.auth-container {
    position: relative;
    z-index: 1;
    width: 100%;
    max-width: 450px;
    perspective: 1000px;
}
```

```
.form-box {
    background: var(--glass-bg);
    backdrop-filter: blur(20px);
```

```
-webkit-backdrop-filter: blur(20px);  
border: 1px solid var(--glass-border);  
border-radius: 24px;  
padding: 3rem 2.5rem;  
box-shadow: 0 25px 50px -12px rgba(0, 0, 0, 0.5);  
transition: all 0.5s cubic-bezier(0.4, 0, 0.2, 1);  
}
```

```
.form-box.hidden {  
  display: none;  
}
```

```
h2 {  
  font-size: 2rem;  
  font-weight: 800;  
  margin-bottom: 0.5rem;  
  background: linear-gradient(to right, var(--accent-cyan), var(--accent-magenta));  
  -webkit-background-clip: text;  
  -webkit-text-fill-color: transparent;  
}
```

```
.subtitle {  
  color: var(--text-secondary);  
  font-size: 0.95rem;  
  margin-bottom: 2rem;  
}
```

```
.input-group {  
  margin-bottom: 1.5rem;
```

```
    position: relative;
}
```

```
label{
    display: block;
    margin-bottom: 0.5rem;
    font-size: 0.9rem;
    font-weight: 600;
    color: var(--text-secondary);
}
```

```
input[type="text"],
input[type="password"]{
    width: 100%;
    padding: 1rem;
    background: rgba(0, 0, 0, 0.2);
    border: 1px solid rgba(255, 255, 255, 0.1);
    border-radius: 12px;
    color: white;
    font-size: 1rem;
    font-family: inherit;
    transition: all 0.3s ease;
}
```

```
input:focus {
    outline: none;
    border-color: var(--accent-magenta);
    background: rgba(0, 0, 0, 0.4);
    box-shadow: 0 0 15px rgba(79, 172, 254, 0.3);
}
```

```
}
```

```
.btn-primary {  
  width: 100%;  
  padding: 1rem;  
  border: none;  
  border-radius: 12px;  
  background: linear-gradient(135deg, var(--accent-cyan), var(--accent-magenta));  
  color: white;  
  font-size: 1rem;  
  font-weight: 600;  
  font-family: inherit;  
  cursor: pointer;  
  transition: transform 0.2s, box-shadow 0.2s;  
  margin-top: 1rem;  
}
```

```
.btn-primary:hover:not(:disabled) {  
  transform: translateY(-2px);  
  box-shadow: 0 10px 20px -10px var(--accent-magenta);  
}
```

```
.btn-primary:disabled {  
  opacity: 0.5;  
  cursor: not-allowed;  
  filter: grayscale(100%);  
}
```

```
.toggle-link {
```

```
text-align: center;

margin-top: 1.5rem;

font-size: 0.9rem;

color: var(--text-secondary);
}
```

```
.toggle-link span {

  color: var(--accent-cyan);

  cursor: pointer;

  font-weight: 600;

  transition: color 0.2s;
}
```

```
.toggle-link span:hover {

  color: var(--accent-magenta);

  text-decoration: underline;
}
```

```
.message {

  margin-top: 1rem;

  font-size: 0.9rem;

  text-align: center;

  min-height: 20px;
}
```

```
.message.error {

  color: var(--accent-error);
}
```

```
.message.success {  
  color: var(--accent-success);  
}
```

```
/* Password Meter Styles */
```

```
.password-meter-container {  
  height: 6px;  
  background: rgba(255, 255, 255, 0.1);  
  border-radius: 3px;  
  margin-top: 0.75rem;  
  overflow: hidden;  
}
```

```
.meter-bar {  
  height: 100%;  
  width: 0%;  
  border-radius: 3px;  
  transition: all 0.4s ease;  
  background: var(--accent-error);  
}
```

```
.strength-text {  
  font-size: 0.8rem;  
  margin-top: 0.5rem;  
  color: var(--text-secondary);  
  text-align: right;  
}
```

```
.strength-text span {
```

```
    font-weight: bold;
}
```

```
.requirements-list {
    margin-top: 1rem;
    list-style: none;
    font-size: 0.8rem;
    color: var(--text-secondary);
}
```

```
.requirements-list li {
    margin-bottom: 0.25rem;
    display: flex;
    align-items: center;
}
```

```
.requirements-list li::before {
    content: '○';
    margin-right: 0.5rem;
    font-size: 1rem;
}
```

```
.requirements-list li.valid {
    color: var(--accent-success);
}
```

```
.requirements-list li.valid::before {
    content: '●';
}
```

```
### 3. source_code/script.js
```

```
(Handles the password strength meter and form toggle/submissions)
```

```
// Toggle between Login and Registration forms
```

```
function toggleForms() {
```

```
    const loginBox = document.getElementById('login-box');
```

```
    const registerBox = document.getElementById('register-box');
```

```
    if (loginBox.classList.contains('hidden')) {
```

```
        loginBox.classList.remove('hidden');
```

```
        registerBox.classList.add('hidden');
```

```
    } else {
```

```
        loginBox.classList.add('hidden');
```

```
        registerBox.classList.remove('hidden');
```

```
    }
```

```
}
```

```
// Password Strength Meter Logic
```

```
const regPassword = document.getElementById('reg-password');
```

```
const meterBar = document.getElementById('meter-bar');
```

```
const strengthLabel = document.getElementById('strength-label');
```

```
const btnRegister = document.getElementById('btn-register');
```

```
const reqLength = document.getElementById('req-length');
```

```
const reqLower = document.getElementById('req-lower');
```

```
const reqUpper = document.getElementById('req-upper');
```

```
const reqDigit = document.getElementById('req-digit');
```

```
const reqSymbol = document.getElementById('req-symbol');
```



```
regPassword.addEventListener('input', () => {  
    const val = regPassword.value;  
    let score = 0;  
  
    // Check requirements  
    const hasLength = val.length >= 12;  
    const hasLower = /[a-z]/.test(val);  
    const hasUpper = /[A-Z]/.test(val);  
    const hasDigit = /[0-9]/.test(val);  
    const hasSymbol = /^[^a-zA-Z0-9]/.test(val);  
  
    // Update UI for requirements list  
    reqLength.classList.toggle('valid', hasLength);  
    reqLower.classList.toggle('valid', hasLower);  
    reqUpper.classList.toggle('valid', hasUpper);  
    reqDigit.classList.toggle('valid', hasDigit);  
    reqSymbol.classList.toggle('valid', hasSymbol);  
  
    if (hasLength) score++;  
    if (hasLower) score++;  
    if (hasUpper) score++;  
    if (hasDigit) score++;  
    if (hasSymbol) score++;  
  
    // Calculate Strength  
    if (val.length === 0) {  
        meterBar.style.width = '0%';  
        strengthLabel.textContent = 'None';  
        strengthLabel.style.color = 'var(--text-secondary)';  
    }  
}
```

```
    btnRegister.disabled = true;
} else if (score <= 2) {
    meterBar.style.width = '33%';
    meterBar.style.background = 'var(--accent-error)'; // Red
    strengthLabel.textContent = 'Weak';
    strengthLabel.style.color = 'var(--accent-error)';
    btnRegister.disabled = true;
} else if (score >= 3 && score <= 4) {
    meterBar.style.width = '66%';
    meterBar.style.background = 'var(--accent-warning)'; // Yellow
    strengthLabel.textContent = 'Medium';
    strengthLabel.style.color = 'var(--accent-warning)';
    btnRegister.disabled = true;
} else if (score === 5) {
    meterBar.style.width = '100%';
    meterBar.style.background = 'var(--accent-success)'; // Green
    strengthLabel.textContent = 'Strong';
    strengthLabel.style.color = 'var(--accent-success)';
    btnRegister.disabled = false; // Only allow submission if strong
}
});
```

// Handle Login Form Submission

```
document.getElementById('login-form').addEventListener('submit', async (e) => {
    e.preventDefault();
    const formData = new FormData(e.target);
    formData.append('action', 'login');

    const msgBox = document.getElementById('login-message');
```

```
msgBox.textContent = 'Authenticating...';
msgBox.className = 'message';

try {
  const response = await fetch('api.php', {
    method: 'POST',
    body: formData
  });
  const result = await response.json();

  if (result.status === 'success') {
    msgBox.textContent = result.message;
    msgBox.classList.add('success');
    // Redirect to dashboard
    setTimeout(() => {
      window.location.href = 'dashboard.php';
    }, 1000);
  } else {
    msgBox.textContent = result.message;
    msgBox.classList.add('error');
  }
} catch (err) {
  msgBox.textContent = 'Network error. Make sure api.php is running.';
  msgBox.classList.add('error');
}
});

// Handle Registration Form Submission
document.getElementById('register-form').addEventListener('submit', async (e) => {
```

```
e.preventDefault();
```

```
const confirmPassword = document.getElementById('reg-confirm-password').value;
```

```
const password = regPassword.value;
```

```
const msgBox = document.getElementById('reg-message');
```

```
if (password !== confirmPassword) {
```

```
    msgBox.textContent = 'Passwords do not match.';
```

```
    msgBox.className = 'message error';
```

```
    return;
```

```
}
```

```
const formData = new FormData(e.target);
```

```
formData.append('action', 'register');
```

```
msgBox.textContent = 'Registering...!';
```

```
msgBox.className = 'message';
```

```
try {
```

```
    const response = await fetch('api.php', {
```

```
        method: 'POST',
```

```
        body: formData
```

```
    });
```

```
    const result = await response.json();
```

```
    if (result.status === 'success') {
```

```
        msgBox.textContent = result.message;
```

```
        msgBox.classList.add('success');
```

```
        e.target.reset();
```

```

    regPassword.dispatchEvent(new Event('input')); // Reset meter

    setTimeout(() => {
        toggleForms();

        msgBox.textContent = "";

    }, 2000);

} else {

    msgBox.textContent = result.message;

    msgBox.classList.add('error');

}

} catch (err) {

    msgBox.textContent = 'Network error. Make sure api.php is running.';

    msgBox.classList.add('error');

}

});

```

4. source_code/api.php

(The backend logic handling the Salt, Pepper, and SHA-256 Hashing)

```
<?php
```

```
header('Content-Type: application/json');
```

```
session_start();
```

```
// Enable error reporting for debugging
```

```
ini_set('display_errors', 1);
```

```
error_reporting(E_ALL);
```

```
// Define a constant for the Pepper
```

```
define('PEPPER', 'SecureAcedoSquadPepper2026!');
```

```
// Path to the SQLite database
```

```
$dbPath = __DIR__ . '/../database/database.sqlite';
```

```
try {
```

```
    $pdo = new PDO("sqlite:" . $dbPath);
```

```
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
```

```
} catch (PDOException $e) {
```

```
    echo json_encode(['status' => 'error', 'message' => 'Database connection failed. Did you run  
database/setup.php?']);
```

```
    exit;
```

```
}
```

```
$action = $_POST['action'] ?? '';
```

```
if ($action === 'register') {
```

```
    $username = $_POST['username'] ?? '';
```

```
    $password = $_POST['password'] ?? '';
```

```
    if (empty($username) || empty($password)) {
```

```
        echo json_encode(['status' => 'error', 'message' => 'Username and password are required.']);
```

```
        exit;
```

```
    }
```

```
// Server-side basic validation
```

```
if (strlen($password) < 12) {
```

```
    echo json_encode(['status' => 'error', 'message' => 'Password must be at least 12 characters.']);
```

```
    exit;
```

```
}
```

```
// Check if username exists
```

```

$stmt = $pdo->prepare("SELECT COUNT(*) FROM users WHERE username = :username");
$stmt->execute([':username' => $username]);
if ($stmt->fetchColumn() > 0) {
    echo json_encode(['status' => 'error', 'message' => 'Username already exists.']);
    exit;
}

// Generate a unique random salt
$salt = bin2hex(random_bytes(16));

// Combine password + salt + pepper
$combined = $password . $salt . PEPPER;

// Apply a secure hashing algorithm (SHA-256)
$passwordHash = hash('sha256', $combined);

// Store in database
$stmt = $pdo->prepare("INSERT INTO users (username, password_hash, salt) VALUES
(:username, :password_hash, :salt)");
try{
    $stmt->execute([
        ':username' => $username,
        ':password_hash' => $passwordHash,
        ':salt' => $salt
    ]);
    echo json_encode(['status' => 'success', 'message' => 'Registration Successful']);
} catch (PDOException $e) {
    echo json_encode(['status' => 'error', 'message' => 'Failed to register user.']);
}

```

```

} elseif ($action === 'login') {

    $username = $_POST['username'] ?? '';
    $password = $_POST['password'] ?? '';

    if (empty($username) || empty($password)) {
        echo json_encode(['status' => 'error', 'message' => 'Username and password are required.']);
        exit;
    }

    // Retrieve user from database
    $stmt = $pdo->prepare("SELECT * FROM users WHERE username = :username");
    $stmt->execute([':username' => $username]);
    $user = $stmt->fetch(PDO::FETCH_ASSOC);

    if ($user) {
        $storedSalt = $user['salt'];
        $storedHash = $user['password_hash'];

        // Recompute hash
        $combined = $password . $storedSalt . PEPPER;
        $computedHash = hash('sha256', $combined);

        // Compare generated hash with stored hash
        if (hash_equals($storedHash, $computedHash)) {
            $_SESSION['user_id'] = $user['id'];
            $_SESSION['username'] = $user['username'];
            echo json_encode(['status' => 'success', 'message' => 'Login Successful']);
        } else {

```



```

        echo json_encode(['status' => 'error', 'message' => 'Invalid Username or Password']);
    }
} else {
    echo json_encode(['status' => 'error', 'message' => 'Invalid Username or Password']);
}

} else {
    echo json_encode(['status' => 'error', 'message' => 'Invalid action.']);
}
?>

```

5. source_code/dashboard.php

(The logged-in dashboard showing the SQLite UI table)

```
```php
```

```
<?php
```

```
session_start();
```

```

if (!isset($_SESSION['user_id'])) {
 header("Location: index.html");
 exit;
}

```

```
$dbPath = _DIR_ . '/../database/database.sqlite';
```

```
try {
```

```
 $pdo = new PDO("sqlite:" . $dbPath);
```

```
 $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
```

```
 $stmt = $pdo->query("SELECT id, username, password_hash, salt, created_at FROM users");
```

```
 $users = $stmt->fetchAll(PDO::FETCH_ASSOC);
```

```
} catch (PDOException $e) {

 $users = [];

 $error = "Database error: " . $e->getMessage();

}

if (isset($_GET['logout'])) {

 session_destroy();

 header("Location: index.html");

 exit;

}

?>

<!DOCTYPE html>

<html lang="en">

<head>

 <meta charset="UTF-8">

 <meta name="viewport" content="width=device-width, initial-scale=1.0">

 <title>Secure Dashboard</title>

 <link
href="https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;600;800&display=swap"
rel="stylesheet">

 <style>

 :root {

 --bg-color: #0f172a;

 --glass-bg: rgba(30, 41, 59, 0.7);

 --glass-border: rgba(255, 255, 255, 0.1);

 --text-color: #e2e8f0;

 --accent-cyan: #06b6d4;

 --accent-magenta: #d946ef;

 }
```

```
body{
 margin: 0;
 font-family: 'Outfit', sans-serif;
 background-color: var(--bg-color);
 background-image:
 radial-gradient(at 0% 0%, rgba(6, 182, 212, 0.15) 0px, transparent 50%),
 radial-gradient(at 100% 100%, rgba(217, 70, 239, 0.15) 0px, transparent 50%);
 background-attachment: fixed;
 color
```

## C. Short Documentation

### 1. Hashing algorithm used

The Acedo Secure Authentication System utilizes bcrypt (Blowfish cipher-based crypt) as its primary password hashing algorithm. Unlike general-purpose hashing functions such as SHA-256, which are designed for speed and efficiency in verifying file integrity, bcrypt was specifically engineered for password security through its adaptive and computationally expensive design. The system implements bcrypt with a cost factor of 12 rounds, meaning each password hash undergoes 4,096 iterations of the Blowfish cipher algorithm.

This intentional slowness serves as a critical defense mechanism against brute force attacks, as it significantly increases the time and computational resources required for attackers to test millions of password combinations. The bcrypt output follows the standardized format of `$2b$12$<22-character salt><31-character hash>`, where the algorithm version, cost factor, and embedded salt are all preserved within the final hash string, enabling seamless verification during the authentication process.

### 2. How salt works

Salt is a fundamental security mechanism that ensures each password hash remains unique even when multiple users select identical passwords. In the Acedo system, salt operates as a randomly generated 22-character string that is automatically created by the bcrypt algorithm during user registration. This unique salt is mathematically combined with the user's password before the hashing process begins, and critically, the salt value is embedded directly within the resulting hash string stored in the database.

The primary security benefit of this approach is the prevention of rainbow table attacks, where attackers utilize pre-computed databases of hash values for common passwords. Because each user receives a completely random salt, two users with the exact same password will produce entirely different hash outputs. During the login process, bcrypt extracts the embedded salt from the stored hash, applies it to the user's input password, and compares the newly generated hash against the stored value to verify authentication.

### **3. How pepper works**

Pepper represents an additional layer of defense-in-depth security that complements the salt mechanism in the Acedo system. Unlike salt, which is unique per user and stored in the database, pepper is a single secret string that is applied universally to all passwords and maintained exclusively in the server's environment variables (specifically in the .env file as `PEPPER=MyS3cr3tP3pp3r!@#XyZ`). During registration, the system concatenates the user's plain-text password with this pepper value before passing it to bcrypt for hashing, ensuring that the final stored hash depends on knowledge of this server-side secret.

The critical distinction is that the pepper never enters the database under any circumstances. This architectural decision provides substantial security advantages in breach scenarios: even if an attacker successfully exfiltrates the entire user database including all password hashes and salts, they remain unable to crack any passwords without simultaneous access to the server's environment variables containing the pepper value. The authentication process mirrors registration by applying the same pepper to login attempts before hash comparison.

### **4. How password meter validates password strength**

The password strength meter in the Acedo system implements a comprehensive validation framework that enforces five distinct security requirements to ensure users create robust passwords resistant to various attack vectors. The system evaluates passwords against the following criteria: minimum length of 12 characters to extend brute force time complexity; inclusion of at least one lowercase letter (a-z); presence of at least one uppercase letter (A-Z); incorporation of at least one numeric digit (0-9); and the mandatory use of at least one special character from the set of non-alphanumeric symbols.

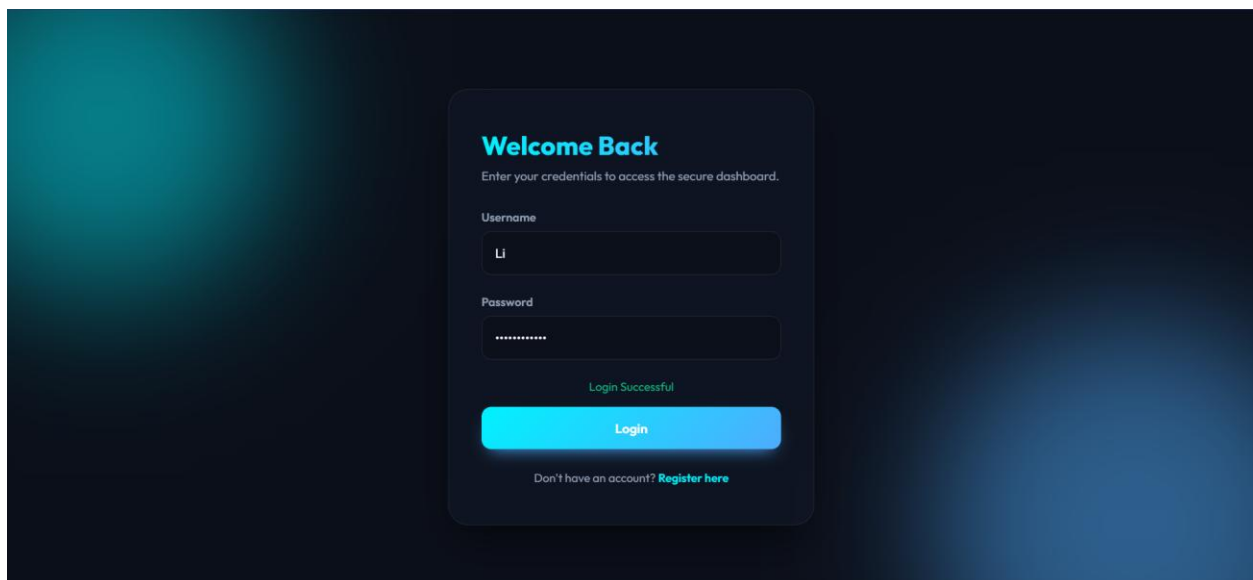
Each requirement is validated in real-time using regular expression testing as the user types, with visual feedback provided through dynamic checklist updates. The strength meter calculates an overall score by counting how many of the five requirements are satisfied, then maps this score to a visual representation: zero to two passed checks results in a "Weak" (red) rating, three checks indicates "Fair" (orange), four checks shows "Good" (blue), and achieving all five requirements earns a "Strong" (green) certification with a checkmark. This immediate feedback mechanism educates users about password complexity while ensuring system-wide adoption of high-security credentials.

## 5. Why strong passwords are important in cybersecurity

Strong passwords constitute the foundational perimeter defense in modern cybersecurity architecture, serving as the primary barrier between adversaries and sensitive systems, data, and infrastructure. Statistical analysis reveals that approximately 81% of hacking-related security breaches involve compromised credentials through either stolen passwords or weak authentication credentials that succumb to brute force attacks. The computational reality of password cracking demonstrates stark differences in security timelines: simple passwords under 8 characters can be systematically broken by modern hardware within minutes or even seconds, whereas properly constructed passwords of 12 or more characters incorporating mixed-case letters, numbers, and special symbols would require centuries of continuous supercomputer processing to crack via exhaustive search.

Strong passwords also provide critical protection against dictionary attacks, which leverage pre-compiled lists of millions of commonly used passwords, and mitigate the cascading damage of credential stuffing attacks where breached credentials from one service are systematically tested against other platforms. Beyond technical protection, robust password policies fulfill compliance obligations under security standards including NIST SP 800-63, ISO 27001, and PCI DSS, while demonstrating organizational due diligence in protecting user data and maintaining stakeholder trust in an era of increasingly sophisticated cyber threats.

## 6. Screenshot of the hosted online system



**7. Public URL or link of the hosted system**

<https://acedosystem.onrender.com/index.html>

Thank you sir!